Fabric Defect Detection: Roboflow vs. a Custom PyTorch CNN

Assignment 1 · code: github.com/ErsatzHitman/fabric-defect-identification

1. Task and data

The goal is to classify photographs of moving fabric as **defective** or **defect_free**. The images are 2448×2048 RGB frames from a fixed camera over several fabrics (blue, lilac, mauve), captured about one second apart. 5,025 frames were sorted into Defected, Defect Free and Wasted using the course procedure: 1,699 frames (batch 1) with a rule-based white-mark detector checked against 48 hand labels, and 3,326 frames (batch 2) by hand. Wasted frames (creases with no defect, the instructor's hand, fabric joints; 98 images) are excluded because the task has two classes. 26 byte-identical duplicates were also removed, so no test image is an exact copy of a training image. That leaves **4,901 images: 3,490 defective (71%) and 1,411 defect_free (29%)**.

	defective	defect_free	total
Train (70%)	2,440	991	3,431
Valid (20%)	698	282	980
Test (10%)	352	138	490
Unseen sample for infer.py	36	14	50

The 70/20/10 split was made once, in Roboflow, and exported as-is to train the CNN, so **both models see exactly the same train, validation and test images**. The 50 images for *infer.py* were drawn at random (stratified by class, fixed seed) from the test split, which neither model trains on. Accuracy is reported on both the 50-image sample and the full 490-image test split; the larger set gives the more reliable comparison.

2. The two pipelines

	Roboflow (no-code)	Custom CNN (PyTorch)
Model	ViT-Base/16, ImageNet-21k pretrained	ResNet18, ImageNet-1k pretrained
Input	512x512 version, resized to 224 by the model	512x512 (tiny defects vanish at 224)
Augmentation	Flip H+V, brightness ±15% (2 copies / image)	Same exported copies + online flip, brightness/contrast jitter
Training	Roboflow hosted GPU, 16 epochs	RTX 5060 laptop GPU, 8 epochs, AdamW, one-cycle LR, class-weighted loss
Inference	Hosted REST API (infer.py)	Local GPU (cnn/predict_cnn.py)

Both pipelines use transfer learning from ImageNet weights, which suits 3,431 training images. The main design difference is resolution. Roboflow's classifier works at 224×224. For the CNN we kept 512×512, because many defects are a few pixels wide in the original frame and fade when the frame is shrunk about tenfold. Class imbalance (71/29) is handled in the CNN with an inverse-frequency weighted loss; Roboflow exposes no such setting.

3. Results

Metric	Roboflow ViT-B/16	ResNet18 (PyTorch)
Accuracy, full test split (n=490)	85.7%	86.9%
Accuracy, 50 unseen images (infer.py)	88.0%	92.0%
Recall, defective	90.1%	86.6%
Recall, defect_free	74.6%	87.7%
Precision, defective	90.1%	94.7%
F1, defective	0.901	0.905
Training time (wall clock)	14 min (incl. queue)	5.7 min
Latency per image, end to end (median, n=50)	364 ms (network API)	28 ms (decode + GPU)
Latency, model forward pass only	n/a (hosted)	3.2 ms

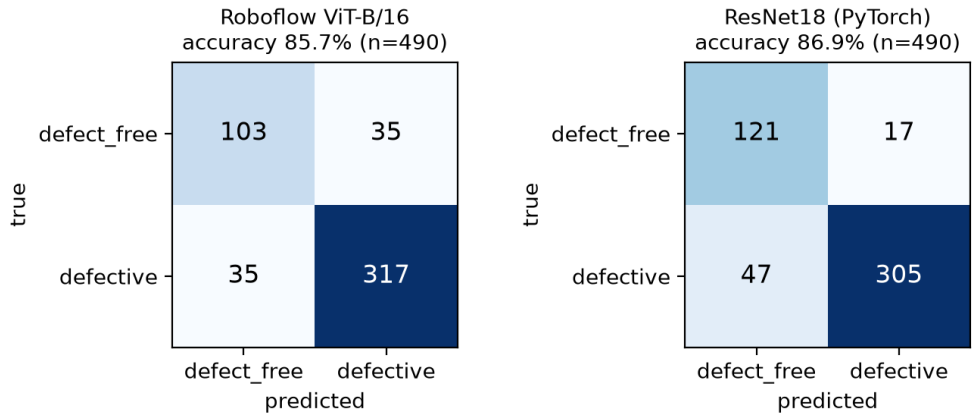


Figure 1. Confusion matrices on the 490-image test split.

Accuracy. The fine-tuned ResNet18 is slightly more accurate: 86.9% vs. 85.7% on the 490-image test split, and 92% vs. 88% on the 50 unseen images. The two models fail differently (Figure 1). Roboflow's ViT leans towards the majority class: it finds 90.1% of defects but misses a quarter of the clean frames (defect_free recall 74.6%). The class-weighted CNN is balanced (86.6% / 87.7%) and is more precise when it says *defective* (94.7% vs. 90.1%). The F1 for the defective class is almost the same (0.905 vs. 0.901). **Training time.** Roboflow took 14.2 min of wall-clock time for 16 epochs on its servers, with no code; its validation accuracy peaked at epoch 4 (86.5%) and then drifted down, a sign of over-fitting that the no-code setup gives no way to stop early. The CNN trained in 5.7 min for 8 epochs on a laptop RTX 5060 GPU and kept the best-validation checkpoint (87.1%). **Latency.** The hosted API needs a median 364 ms per image, almost all of it network round-trip. The local CNN takes 28 ms end to end, including JPEG decoding, and 3.2 ms for the forward pass alone: about 13× faster than the API.

Trade-off. Roboflow wins on effort: upload, split, augmentation, training and a deployed REST endpoint took minutes, with no ML code or GPU, and the model is immediately usable from any device. The custom CNN wins on control and cost per prediction. It allows the input resolution, class weighting and checkpoint selection that made it more balanced, and it runs offline at a fraction of the latency, which matters on a production line that inspects many frames per second.

4. Failure analysis of misclassified edge cases

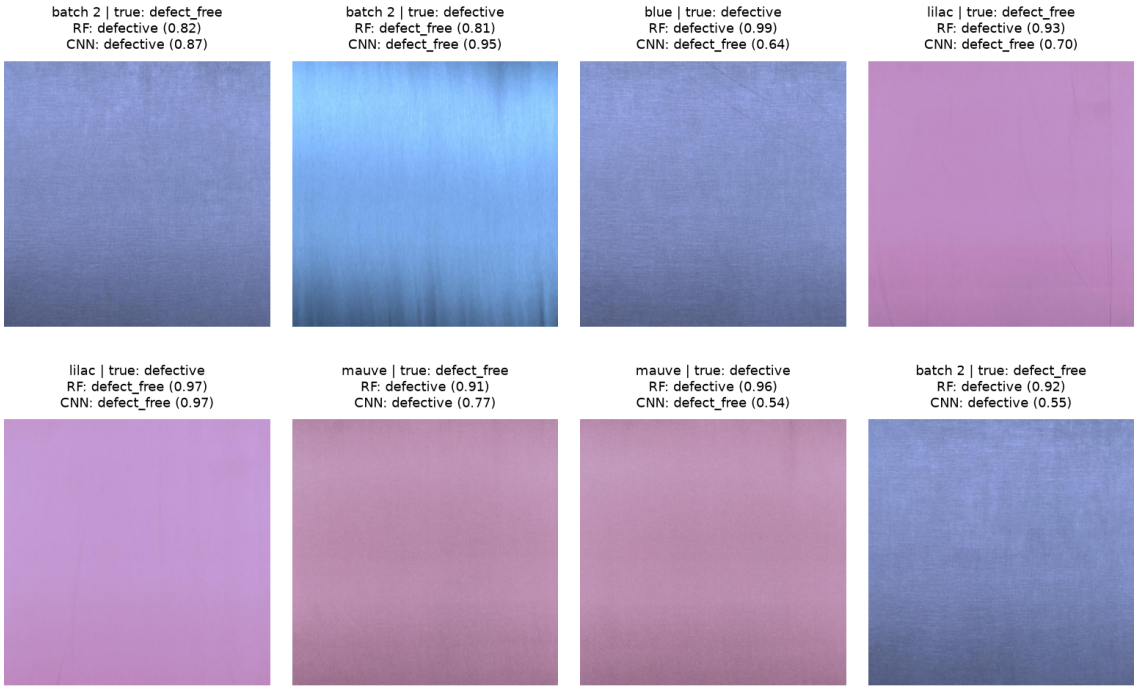


Figure 2. Test images misclassified by at least one model (RF = Roboflow; confidence in brackets).

Of the 490 test images, 90 were misclassified by at least one model and **44 by both**. Those 44 point mostly at the data rather than at either architecture. **(1) Inconsistent labels for the same fabric.** In batch 1 every blue frame was labelled defective, because each had at least one faint white slub; in batch 2 many near-identical blue frames were labelled defect_free (Figure 2, first and last panels). Both models call these defective with 0.8–1.0 confidence: they are not really wrong, the labels disagree. Of the 16 defect_free test images that both models missed, 15 come from batch 2, and those inspected are of this kind. **(2) Fabric colour used as a shortcut.** Almost all lilac training frames are defect_free, so the CNN learned “lilac means clean”: it missed 11 of the 12 defective lilac test frames, at up to 0.97 confidence, and Roboflow missed 9 of them. Those defects are faint, thin white lines that barely differ from the smooth fabric (fifth panel). **(3) Faint marks near the decision boundary.** On grainy mauve and on blue, short faint white marks look like normal texture. The CNN missed 8 such defects, all with low confidence (0.50–0.69), while Roboflow caught every one: the CNN's class weighting tips borderline frames towards defect_free. A confidence threshold (e.g. send anything below 0.7 to a human) would catch these. **(4) Creases and lighting.** Wrinkle edges and the darker band at the bottom of each frame (vignetting) trigger false defects in clean lilac frames, mostly in Roboflow's 224-px model. **(5) Acquisition differences.** Batch 2 includes a lighter, streaky blue fabric shot under different conditions (second panel); both models misread it. Finally, frames are about one second apart, so neighbouring frames of the same defect can land in train and test. Test accuracy may therefore be slightly optimistic for truly new fabric.

5. Conclusion

On identical data, the fine-tuned ResNet18 matched and slightly beat Roboflow's hosted ViT (86.9% vs. 85.7%), trained faster (5.7 vs. 14.2 min) and predicted 13× faster locally. Roboflow remains the quicker way to a deployed model with no code or GPU. Both reach a ceiling near 86–87%, set more by the data than the model: labels that disagree between labellers, fabric colour tied to class, and defects only a few pixels wide. The most effective next steps are (i) re-labelling the conflicting blue frames under one written rule, (ii) adding defective lilac and clean blue/mauve examples so colour stops predicting the class, (iii) training on 512-px crops or tiles so faint marks keep their pixels, and (iv) splitting by capture time rather than at random.